

Signet 2: intents, archetypes and translation profiles

How different games can play together without translating every game into every other game.

Status: **draft proposal** (not implemented) · Document version: 0.1 · Date: October 5, 2026

Project: **Signet Protocol** · github.com/kian-cx/signetprotocol · signetprotocol.io · Apache-2.0 license

One-minute summary

- **Today**, for two games to play together you have to translate one into the other, piece by piece. Every new game adds more work.
- **Signet 2** proposes something else: each game translates **into a shared meaning** and **out of a shared meaning**. The server only understands meaning: what the player wants to do (*intents*) and what exists in the world (*archetypes*).
- Each game keeps what makes it unique: its interface, its mechanics, its animations. It only decides **how to show** each archetype and **which button means what**.
- Before playing, the player **sets up their controls** once (like in any game). That solves everything obvious without artificial intelligence.
- For what is not obvious, an **AI model picks from a closed list of options** and a person confirms. The decision is saved and from then on it is always the same.
- **Signet Protocol** is the standard (protocol, SDK and standard translator). **Signet Forge** is a separate tool to create, review and improve translations on top of that standard.

1. The problem

A game is much more than its world: it has its own controls, its own items, its own way of moving and shooting. When we want a Minecraft player and a Doom player in the same match, we have to answer questions like these:

- Doom has a pistol. Minecraft does not. What is the Minecraft player holding?
- A modern game lets you jump, crouch and slide. An old one does not. What happens when someone jumps?
- Every engine sends "move forward" in a different way. How does the server know it is the same thing?

If we solve this **pair by pair** (Doom↔Minecraft, Doom↔OpenArena, Minecraft↔OpenArena...), 10 games make 45 pairs and 20 games make 190. It does not scale, and every pair piles up special cases.

2. The idea in one sentence

Do not translate from game to game. Translate from game to meaning, and from meaning to game.

This way each new game only needs **one** translator (to and from the shared meaning), and it can automatically play with all the others.

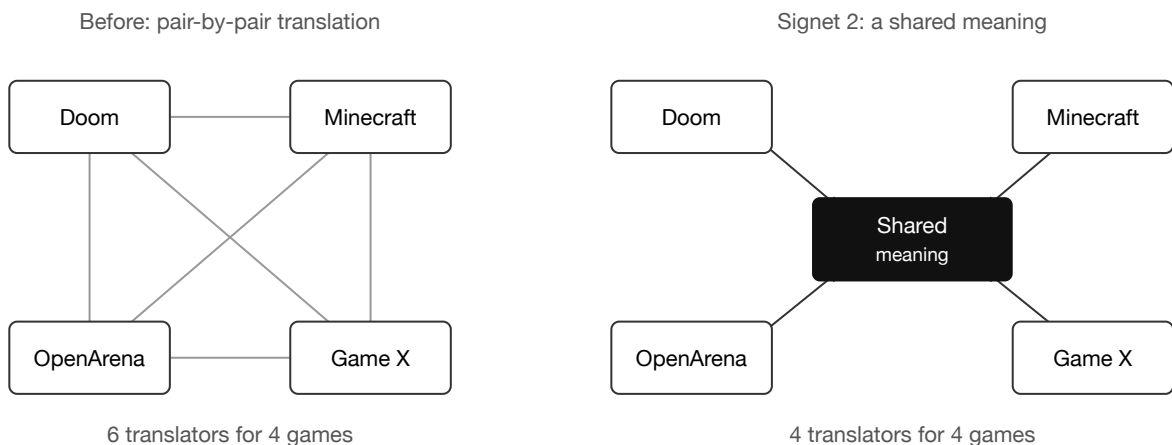


Figure 1. With a shared meaning, the number of translators grows at the same rate as the number of games.

3. Overall architecture

The system has three parts. What matters is **what stays inside each game** and **what travels over the network**.

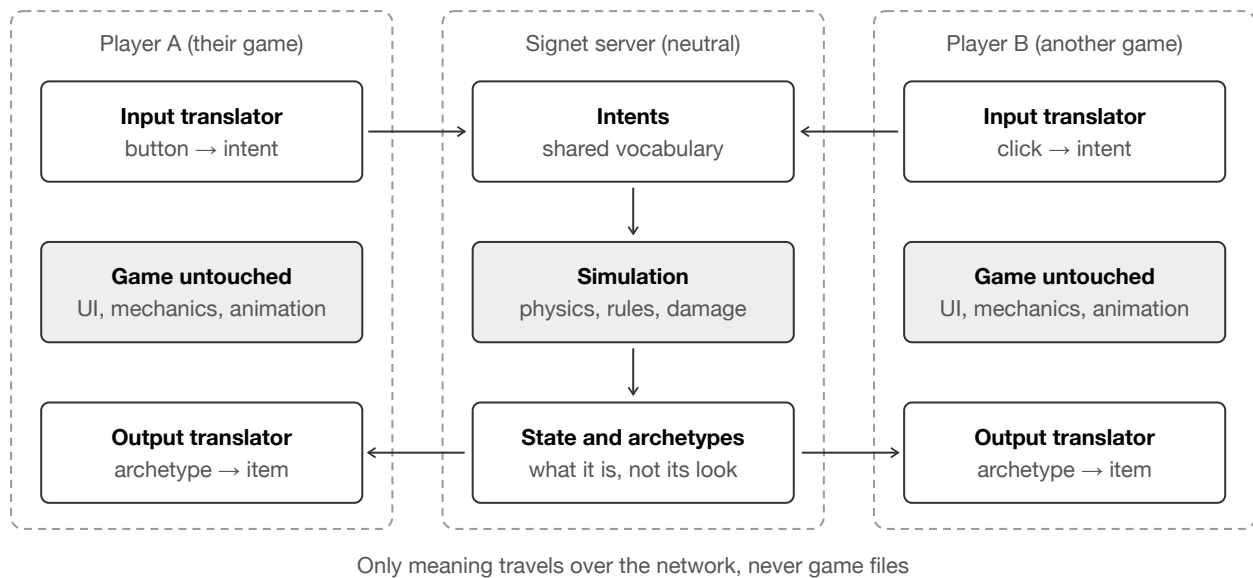


Figure 2. Signet 2 architecture. Intents go up; state and archetypes come down.

1. **The player** presses a key in their game. The **input translator** turns it into an intent ("move forward", "fire").
2. **The server** receives everyone's intents, applies the rules and the physics, and decides what happens: positions, hits, damage, deaths. It is the only authority, so nobody can cheat by modifying their client.
3. The server sends back the **state** and the **archetypes** ("there is a ranged weapon here"). It does not say what it looks like.
4. Each game's **output translator** chooses how to show it with what that game has.

4. The concepts, one by one

Concept	What it is	Example
Intent	What the player <i>wants</i> to do, regardless of which key they pressed or which game they use.	move, turn, jump, fire, use, switch weapon
Intent vocabulary	The shared list of every intent the standard recognizes. Published by the SDK.	~15 intents in version 0
Archetype	Something that <i>exists</i> in the world, described by its function, with no shape or graphics.	weapon.ranged, weapon.melee, health_pickup, armor
Archetype catalog	The shared list of archetypes. Also published by the SDK.	~15 archetypes in version 0
Appearance palette	The items <i>one</i> game has that can stand for an archetype. Each game's "bag of items".	Minecraft: bow, crossbow, fishing rod
Capabilities	What a game declares when it connects: which intents it can send and which archetypes it can show.	"I cannot jump", "I can show weapons and health pickups"
Resolver	The piece that picks the best option from a closed list. It can be a table, a person or an AI model.	"weapon.ranged in Minecraft → bow"
Translation profile	The file where decisions are saved. Once pinned, a decision does not change.	signet.lock.json

One archetype, three appearances

The server says	Minecraft shows	Doom shows	OpenArena shows
weapon.ranged	a bow	the pistol, with its animation	the machine gun, with its animation
health_pickup	a golden apple	the Doom medikit	the health orb

For the server, all three players have the same thing: a ranged weapon with the same damage and the same range. Only what each one sees on their screen changes.

5. Setting up controls in advance (calibration)

Before the first match, the player goes through a short wizard, just like setting up the controls of a new game. This has a huge advantage: **every obvious decision is made by the player themselves, exactly, without using any model.**

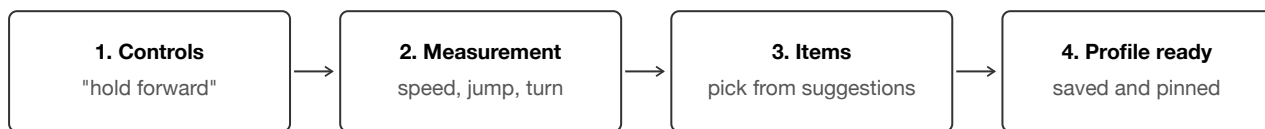


Figure 3. The calibration wizard, before playing.

1. **Controls.** The wizard asks: "hold the key to move forward for two seconds". It records which key the player used and binds it to the *move* intent. The same for strafing, turning, firing and switching weapons.
2. **Measurement.** For some intents, the wizard **measures** how that game behaves: walking and running speed, jump height, turn speed, eye height, maximum step height. We call this the **motion profile**, and it is what makes the translation faithful.
3. **Items.** For each archetype, the items in the game's palette are shown, with the model's suggestion highlighted. The player accepts it or picks another.
4. **Profile ready.** Everything is saved in the translation profile. From here on, the game works with no ambiguity.

Three difficulty tiers

Tier	Examples	How it is solved
0 · Obvious	forward, back, strafe, turn, fire	Set by the player during calibration. Exact, no model.
1 · Measurable	run, jump, crouch	Set up and also measured (motion profile). No model.
2 · Complex	abilities, inventory, emotes, vehicles, unusual items	The model suggests from closed options and a person confirms .

Calibration clears everything obvious out of the way. The model only works on tier 2, where it is actually needed, and the number of decisions it has to make is small.

6. How the model decides

The model does not "invent" translations. It works like someone who is shown a list of options and asked which one fits best. For this we propose a **contrastive ranking model**, for example **CLM-v0.1-8B** (Contrastive-LM, Apache-2.0 license): it receives a situation and a list of candidates, and returns each candidate with a score.

```

Situation:  archetype "weapon.ranged", game Minecraft
Candidates: bow · crossbow · fishing rod · snowball · trident
Result:     (illustrative example, not a real measurement)
            bow 0.81 · crossbow 0.74 · trident 0.55 · snowball 0.31 · fishing rod 0.22
  
```

Advantages of this approach:

- **It cannot go outside the list.** Since it only scores candidates, it never proposes something the game does not have or an intent that does not exist.
- **It is explainable.** You see the scores of every option, not just the winner.
- **It is replaceable.** The resolver is an interface: a model today, a better one tomorrow, or a hand-written table.

Decision order

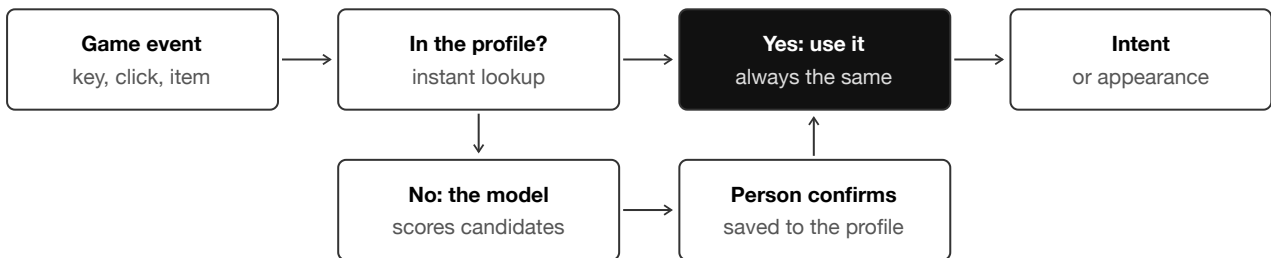


Figure 4. The fast path (top) is used almost always. The model only steps in the first time something new appears.

Priority when there are several sources: **1)** what the player pinned → **2)** the translator's profile → **3)** the model's suggestion → **4)** a safe default.

The model is never in the game loop. The server simulates 20 times per second and a model of this size needs a GPU. So the model works before playing or when something new appears, and its answer is saved. During the match only the profile is consulted, which is instant.

7. Translation profiles: decisions that do not change

Every decision is saved in a file next to the translator, similar to the lock files that package managers use. Once pinned, the decision is **deterministic**: it always gives the same result.

```
{
  "key": "archetype:weapon.ranged",
  "choice": "minecraft:bow",
  "source": "human",
  "candidates": [["minecraft:bow", 0.81], ["minecraft:crossbow", 0.74],
["minecraft:fishing_rod", 0.22]],
  "resolver": "clm-v0.1-8b",
  "pinned": true
}
```

What happens with something that is not in the profile?

Policy	Behavior	For whom
Strict	It is ignored. Nothing happens.	Tournaments, serious servers
Suggest	The model's best option is used, marked as provisional, and queued for review.	Casual matches
Ask	It is decided on the spot with the development tool.	Whoever is building the translator

Safety rule: intents that affect the simulation (moving, firing) must **always** come from a pinned decision. Otherwise client prediction and the server would drift apart. Appearance is local to each player, so it can stay provisional without affecting anyone else.

Because the profile is a text file in the repository, the community can improve any translator with a pull request.

8. What is the standard and what is a separate tool

	Signet Protocol (the standard)	Signet Forge (companion tool)
What it is	The protocol, the SDK and the standard translator .	An application to create, review and improve translations.
Includes	Intent vocabulary, archetype catalog, capabilities, translation API, profile format, calibration wizard, reference server.	Suggestion review, profile editing, pending queue, dataset, model fine-tuning.
Needed to play?	Yes.	No. Only for people who build translators.
Uses AI?	Not during the match: it only reads profiles.	Yes, to suggest and to train.
Goal	To be simple, stable and reliable.	To make translations easy to build and better over time.

Signet Forge: how it works

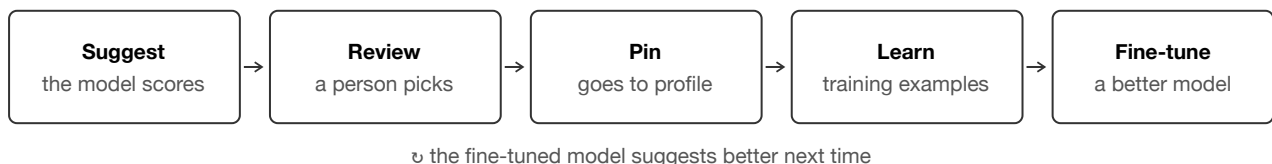


Figure 5. The Signet Forge cycle.

Every approved decision is an example: *situation, options, chosen*. With enough examples the model can be fine-tuned for video games. With a model like CLM, which separates a large frozen encoder from two small "heads", only those heads would need training, which should be cheap.

Data rules:

- Only names and identifiers. **Never** game files: no textures, sounds or models.
- Nothing about the player: no usernames and no key logs.
- Contributed under an open license.

9. Why this makes the protocol more reliable

- **The server does not change for each new game.** It only understands intents and archetypes.
- **Nothing is random during the match.** Every decision comes from a pinned profile.
- **AI stays outside the protocol.** If a model fails or changes, the match is not affected.
- **Limited games do not break the match.** Thanks to capabilities, the server knows what each one can do.
- **Building a translator is easier.** Calibration solves the obvious, the model suggests the hard parts and Forge keeps the work.

10. Limits and rules

- **Legal.** Calibration only reads the player's own devices and what the game officially allows: open-source engines, permitted mods or your own server. No code is injected into online games and no anti-cheat systems are bypassed. A closed game with no legitimate way to read or drive it is out of scope.
- **Physics.** Intents do not remove the need for a common physics on the server. The motion profile helps each game feel faithful, but the server remains the authority.
- **Model not validated.** We have not tested any model for this yet. The figures for the model mentioned are its authors', on other tasks.

11. Compatibility with the current version

Signet/1 already contains the seed of this: its Intencion message carries forward, strafe, turn, run, fire, use and weapon, and the weapons ARMA_PUNO, ARMA_PISTOLA and ARMA_ESCOPETA are already archetypes that each viewer draws its own way. Signet/1 only grows with optional fields, so the first steps (capabilities in the greeting and archetype identifiers) can be added without breaking anything. The full English vocabulary and any renaming are left for Signet/2.

12. Proposed roadmap

#	Step	Outcome
1	Publish version 0 of the vocabulary (~15 intents) and of the catalog (~15 archetypes).	A stable list to discuss
2	Add capabilities to the protocol greeting, as an optional field.	Compatible with Signet/1
3	Define the translation profile format and the calibration wizard.	Tiers 0 and 1 without AI
4	Prototype without AI: "weapon.ranged" → bow in the Minecraft translator.	End-to-end demonstration
5	30-case experiment: hand-written rules vs text similarity vs contrastive model.	Decide whether the model adds value
6	First version of Signet Forge (review, pin, export data).	A tool for the community

13. Open questions

- What is the minimum a motion profile must measure?
- How do we measure it in a game that does not report its position?
- Who maintains the vocabulary and the catalog, and how are new entries accepted?
- Where does the model run: on the player's machine, on a shared service, or only in Signet Forge?
- How are conflicting community profiles for the same game resolved?

Glossary

Archetype	Something that exists in the world, described by its function, without graphics.
Calibration	Setting up controls and measuring before playing.
Capabilities	What a game declares it can send and show.
Deterministic	Always giving the same result for the same situation.
Intent	What the player wants to do.
Appearance palette	The items of a game that can stand for archetypes.
Motion profile	Measurements of how a game moves: speed, jump, turn.
Translation profile	File with a translator's pinned decisions.
Resolver	Whatever picks the best option from a list: a table, a person or a model.
Signet Forge	Companion tool to create, review and improve translations.
Standard translator	The SDK component that converts between a game and the shared meaning.

Signet Protocol · Apache-2.0 · Games and their trademarks belong to their owners. This document is a proposal and may change.